



How To Debug in C

with Rahul Reji

1. Compile-time errors

```
$ gcc joseph.c
exclaim.c: In function 'main':
exclaim.c:12:28: error: passing argument 1 of 'square' makes integer from pointer without a cast [-Wint-conversion]
   12 |         printf("%d \n", square("abc"));
      |                             ^~~~~
      |                             |
      |                             char *
exclaim.c:3:16: note: expected 'int' but argument is of type 'char *'
     3 | int square(int n) {
      |           ~~~~~^
```

This one is a "type error". `square` expects an integer, but got a character pointer instead

```
$ gcc world.c
world.c: In function 'main':
world.c:4:14: error: implicit declaration of function 'malloc' [-Wimplicit-function-declaration]
    4 |         int* x = malloc(50);
      |                   ^~~~~~
world.c:2:1: note: include '<stdlib.h>' or provide a declaration of 'malloc'
    1 | #include <stdio.h>
      | ^
+++ |+#include <stdlib.h>
world.c:4:14: note: include '<stdlib.h>' or provide a declaration of 'malloc'
```

The compiler is quite helpful, it usually tells you how to fix most compile-time errors.



What exactly causes a segmentation fault?

A segmentation fault is a failure caused when a process attempts to access a memory segment that it is not allowed to access.



The GNU Debugger

To run a program via GDB, first compile it using the `-g` flag.

```
$ gcc main.c -g
```

then launch GDB providing the executable as an argument:

```
$ gdb ./a.out
```

github.com/whitespace404/buggy-code

It will look somewhat like this:

```
rahul@ThinkPad-P1:~/Documents/buggy-code$ gdb a.out
GNU gdb (GDB) 17.2
Copyright (C) 2025 Free Software Foundation, Inc.
License GPLv3+: GNU GPL version 3 or later <http://gnu.org/licenses/gpl.html>
This is free software: you are free to change and redistribute it.
There is NO WARRANTY, to the extent permitted by law.
Type "show copying" and "show warranty" for details.
This GDB was configured as "x86_64-pc-linux-gnu".
Type "show configuration" for configuration details.
For bug reporting instructions, please see:
<https://www.gnu.org/software/gdb/bugs/>.
Find the GDB manual and other documentation resources online at:
  <http://www.gnu.org/software/gdb/documentation/>.

For help, type "help".
Type "apropos word" to search for commands related to "word"...
Reading symbols from a.out...
(gdb) █
```

If it says "Reading symbols from a.out", that means you compiled it correctly and are fine to proceed.

If it says "No debugging symbols found", that means you forgot the `-g` flag while compiling.

Setting a breakpoint

A breakpoint pauses your program whenever a particular point in the program is reached. You can decide where this should be. This allows you to check values of intermediary variables while the execution is halted.

Usage:

1. `b [line number]`: Halts execution right before that line
2. `b [function name]`: Halts execution when that function is entered

When GDB stops at "line 5", this means that gdb is currently waiting "between" lines 4 and 5. Line 5 hasn't executed yet. Keep this in mind! You can execute line 5 with the next command, but line 5 has not happened yet.

Where do I put breakpoints?

It comes with practice. While starting out, use `b main` and run through every line of your code. As you begin to get more used to GDB, you'll sort of get an idea of where to put them. For example, instead of `b main`, you can move the breakpoint down until you read input. Or, instead of `main`, choose a function that you suspect isn't working correctly.

Once you have chosen your breakpoint, run it using the `run` command. The program will now start normal execution, and pause when the breakpoint is encountered.

To quit GDB, use `"quit"` or `"q"`

Recap

1. Compile with `gcc -g`
2. Run `gdb a.out`
3. Go for a `layout src`
4. Within the gdb prompt, set a breakpoint with `b`
5. Type `run`



Inspecting variables

The `print` (shorthand `p`) can be used to inspect values of variables. To use-
`print [identifier name]`

It works on basically everything- integers, arrays, strings and structures, and supports C syntax like dereferencing (`*`) and structure pointer dereferencing (`→`) operators

For strings, by default it prints the character array until `\0` is encountered. If you want to print the first `n` characters instead you can use `print mystring@n`

Example: `mystring@50` prints the first 50 characters starting at address `mystring`.



Watchpoints

You can use a watchpoint to stop execution whenever the value of an expression changes, without having to predict a particular place where this may happen.

This is useful when a program gives you weird values and you want to know from where that value was added. You can also set a watch point to an array, not only variables.



Usage:

watch [identifier]: GDB will now break whenever the identifier is written into by the program and its value changes.



Continuing Execution Flow

1. `next` (shorthand: `n`)

Runs the current line and goes to the immediately next line.

2. `step` (shorthand: `s`)

Runs the current line, and if there aren't any function calls in the current line- goes to the next line.

If there are any function calls, goes INTO the function's definition and halts there.

3. `continue` (shorthand: `c`)

Runs until the next breakpoint

Other useful commands

1. `layout src`: Shows the source code alongside the GDB prompt.

This layout is sometimes prone to slight screen glitches. Remember to press `Ctrl+L` to refresh the screen every time something looks unusual.

2. `list`: Shows the source code centered to where you currently are.

3. `info locals`: Shows all variables in the current scope, and their values.

4. `set variable [variable name] = [value]`: Changes the value of a variable on the fly

■ These above commands are enough to help you debug.

Just be thorough with the above commands. It does take some practice knowing when to apply each command. I recommend you try using GDB and get used to how the code moves, by using `step` and `next`. Do it for like 3-5 programs and get addicted to the process of finding bugs, and you'll be silently grinning next time you get a segfault (hopefully). There is some sort of mysterious charm to debugging, and I hope I helped you guys see the beauty in it. Thank you for coming.